

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30723
	Apellidos y nombres del estudiante: Caiza Carlos, Kerly Chiriboga, Alexander Villacrés

BOCATTO V3.0 — Plan de Pruebas Unitarias

Identificador del documento: TP-BOCATTO-UT-001

Norma de referencia: IEEE Std 829-2008 (Standard for Software and System Test Documentation)

Proyecto: BOCATTO V3.0

Tipo de documento: Plan de Pruebas Unitarias

Alcance: Backend (Node.js / Express / Mongoose) y Frontend (SPA en JavaScript puro)

1. Identificador del Plan de Pruebas.....	3
2. Introducción	4
2.1 Propósito.....	4
2.2 Antecedentes.....	4
2.3 Módulos del proyecto bajo prueba	4
2.4 Definiciones, acrónimos y abreviaturas.....	5
2.5 Referencias.....	5
3. Elementos a Probar.....	6
3.1 Elementos del backend (módulos Node.js, ES modules)	6
3.2 Elementos del frontend (Vanilla JS, ES modules).....	6
3.3 Elementos excluidos de las pruebas unitarias (fuera de alcance).....	7
4. Características a Probar	7
4.1 Reglas de validación (BE y FE)	8
4.2 Lógica de dominio — patrón Builder y Director (BE)	8
4.3 Patrón Prototype (BE)	8
4.4 DTO / Mapper (BE)	9
4.5 Reglas de negocio de la capa de servicio (BE)	9
4.6 Middleware (BE)	9
4.7 Utilidad HTTP / respuesta (BE)	10
4.8 Frontend — clases utilitarias (FE)	10

4.9 Frontend — servicios sobre fetch (FE).....	10
4.10 Frontend — bus de eventos y helpers de UI (FE)	11
5. Características que No se Probarán	11
6. Enfoque de Pruebas.....	11
6.1 Niveles de prueba.....	11
6.2 Técnica de prueba.....	12
6.3 Framework de pruebas	12
6.4 Convención de estructura de pruebas	12
6.5 Patrón de diseño de pruebas.....	13
6.6 Estrategia de mocks.....	13
6.7 Objetivos de cobertura de código.....	13
6.8 Gestión de datos de prueba.....	13
6.9 Herramientas e Integración Continua.....	13
7. Criterios de Aprobación/Fallo.....	14
7.1 Aprobación/Fallo por caso de prueba	14
7.2 Aprobación/Fallo por módulo.....	14
7.3 Aprobación/Fallo por componente (backend / frontend)	14
7.4 Aprobación/Fallo global.....	14
8. Criterios de Suspensión y Requisitos de Reanudación.....	14
8.1 Criterios de suspensión (cuándo detener la fase)	14
8.2 Requisitos de reanudación	15
9. Entregables de las Pruebas	15
10. Tareas de Pruebas	15
11. Necesidades del Entorno	16
11.1 Hardware	16
11.2 Software	16
11.3 Red.....	16
11.4 Archivos de configuración a añadir	16
12. Calendario.....	17
13. Riesgos y Contingencias.....	18
13.1 Riesgos del proyecto.....	18
13.2 Esquema de severidad de defectos	18
14. Aprobaciones.....	18
Apéndice A — Especificaciones de Pruebas Unitarias del Backend.....	19
A.1 BE-01 — ApiError (backend/src/utls/ApiError.js).....	19

A.2 BE-02 — ApiResponse (backend/src/utis/ApiResponse.js).....	19
A.3 BE-03..08 — Validadores	20
A.4 BE-09..10 — MesaDTO y MesaMapper	21
A.5 BE-11..13 — Patrón Prototype.....	22
A.6 BE-14..15 — PedidoBuilder y DirectorPedido	23
A.7 BE-16 — MesaService	25
A.8 BE-17 — AuthService.....	26
A.9 BE-18..20 — Middleware.....	28
Apéndice B — Especificaciones de Pruebas Unitarias del Frontend	28
B.1 FE-01 — Storage (frontend/utis/storage.js)	29
B.2 FE-02 — Auth (frontend/utis/Auth.js)	29
B.3 FE-03 — EstadoPedido (frontend/utis/EstadoPedido.js)	30
B.4 FE-04..08 — Validadores del Frontend	30
B.5 FE-09 — MesaService sobre fetch (frontend/services/mesaService.js)	31
B.6 FE-10..13 — api, Toast, Modal, EventBus	33
Apéndice C — Catálogo de Casos de Prueba.....	33
C.1 Casos de prueba del backend	34
C.2 Casos de prueba del frontend	36
Apéndice D — Extensión de Fase 2 (cobertura completa de RFs)	39
D.1 Nuevos elementos backend (BE-21..27).....	39
D.2 Nuevos elementos frontend (FE-14..24).....	39
D.3 Nuevos casos de prueba — Backend (TC-UT-048..085).....	39
D.4 Nuevos casos de prueba — Frontend (TC-UT-136..170).....	42
D.5 Production fixes aplicados en fase 2	44

1. Identificador del Plan de Pruebas

Campo	Valor
<i>ID del documento</i>	TP-BOCATTO-UT-001
<i>Título</i>	BOCATTO V3.0 — Plan de Pruebas Unitarias
<i>Versión</i>	1.0
<i>Estado</i>	Borrador
<i>Autor</i>	Grupo ADS — Líder de QA
<i>Fecha de emisión</i>	2026-07-08
<i>Norma</i>	IEEE Std 829-2008
<i>Planes relacionados</i>	(ninguno — este es el plan maestro de pruebas unitarias del proyecto)

2. Introducción

2.1 Propósito

Este documento define el plan, el enfoque, los recursos y el calendario para las **actividades de pruebas unitarias** del sistema de gestión de restaurante BOCATTO V3.0. Está pensado para servir como **plano de referencia** del proceso de pruebas: el equipo que ejecute las pruebas deberá seguirlo como la referencia autorizada para alcance, técnica, criterios y entregables.

2.2 Antecedentes

BOCATTO V3.0 es una aplicación web full-stack que respalda las operaciones de un restaurante: administración de usuarios y roles, registro de clientes, gestión de mesas, menú, construcción de pedidos, flujo de cocina, caja, promociones, tableros (dashboards) y reportes. Además soporta un flujo público de pedidos por QR.

Nota: este plan se aplica sobre **BOCATTO V3.0**. Respecto a V2.0, V3.0 introduce: campos clienteNombre y observaciones en PedidoBuilder/DirectorPedido; validación ampliada del canal TELEFONO (requiere clienteNombre, telefonoEntrega y direccionEntrega); reescritura de AuthService con bloqueo temporal (bloqueadoHasta, auto-desbloqueo, códigos INTENTOS:N / BLOQUEADO:N); URL base del frontend cambiada a <https://bocatto-2gaz.onrender.com/api/v1>; y renombrado de métodos en EventBus (emitir/escuchar/eliminar) y Modal (abrir/cerrar).

El sistema se entrega como dos componentes desplegables de forma independiente:

- **Backend** — Node.js 20+, Express 5, Mongoose 8, MongoDB, autenticación con JWT, Socket.IO.
- **Frontend** — SPA en JavaScript puro (sin framework de build), estructura modular (servicios, controladores, vistas, validadores, utilidades), plantillas HTML, CSS, cliente de Socket.IO.

2.3 Módulos del proyecto bajo prueba

#	Módulo	Archivos backend	Archivos frontend
1	Autenticación	services/AuthService.js, middlewares/authMiddleware.js, middlewares/roleMiddleware.js	services/authService.js, utils/Auth.js, utils/storage.js
2	Usuarios	services/UsuarioService.js, validators/UsuarioValidator.js, dto/UsuarioDTO.js, mappers/UsuarioMapper.js	services/usuarioService.js, validators/UsuarioValidator.js, views/UsuarioView.js
3	Clientes	services/ClienteService.js, validators/ClienteValidator.js	services/clienteService.js, validators/ClienteValidator.js
4	Mesas	services/MesaService.js, validators/MesaValidator.js, dto/MesaDTO.js, mappers/MesaMapper.js	services/mesaService.js, validators/MesaValidator.js
5	Menú	services/MenuService.js, validators/MenuValidator.js, patterns/prototype/MenuPrototype.js	services/menuService.js, validators/MenuValidator.js
6	Pedidos	services/PedidoService.js, validators/PedidoValidator.js, patterns/builder/PedidoBuilder.js, patterns/builder/DirectorPedido.js, dto/PedidoDTO.js	services/pedidoService.js, validators/PedidoValidator.js, utils/EstadoPedido.js
7	Cocina	services/CocinaService.js	services/cocinaService.js, views/CocinaView.js
8	Caja	services/CajaService.js	services/cajaService.js, views/CajaView.js
9	Promociones	services/PromocionService.js, validators/PromocionValidator.js, patterns/prototype/PromocionPrototype.js	services/promocionesService.js, views/PromocioneView.js
10	Reportes	services/ReporteService.js	services/reporteService.js, views/ReporteView.js
11	Dashboards	services/DashboardService.js	services/dashboardService.js, views/DashboardView.js

#	Módulo	Archivos backend	Archivos frontend
1	Utilidades	utils/ApiError.js, utils/ApiResponse.js	utils/EventBus.js, utils/Toast.js, utils/Modal.js, utils/Form.js, utils/Loader.js, utils/AppState.js, utils/router.js, utils/layout.js, utils/socket.js, utils/Alert.js
2	comunes		

2.4 Definiciones, acrónimos y abreviaturas

Término	Significado
SUT	System Under Test (Sistema Bajo Prueba)
DUT	Design (or class) Under Test (Diseño o clase bajo prueba)
DTO	Data Transfer Object (Objeto de Transferencia de Datos)
API	Application Programming Interface
JWT	JSON Web Token
BC	Boundary Condition (Condición de frontera, p. ej. vacío, nulo, mín., máx.)
EP	Equivalence Partition (Partición de equivalencia)
BVA	Boundary Value Analysis (Análisis de valores límite)
AAA	Arrange / Act / Assert (patrón Preparar / Actuar / Afirmar)
MUT	Module Under Test (Módulo bajo prueba)
CLI	Command Line Interface (Interfaz de línea de comandos)
SPA	Single-Page Application (Aplicación de una sola página)

2.5 Referencias

- IEEE Std 829-2008 — Standard for Software and System Test Documentation.
- BOCATTO V2.0 — Especificación de Requisitos de Software (SRS) v1.0. 1. Elicitacion/1.1 SRS/G2_30723_SRS_ADS_V1.0.pdf
- BOCATTO V2.0 — Historias de Usuario y Backlog. 1. Elicitacion/1.4 Historias de Usuario/, 1. Elicitacion/1.6 Backlog/
- BOCATTO V2.0 — Documento de Diseño de Arquitectura v1 y v2. 2. Diseños/1.1 Diseño de Arquitectura/Arquitectura_Propuesta_G2_V1.pdf, Arquitectura_Propuesta_G2_V2.pdf
- BOCATTO V2.0 — Patrones de Diseño aplicados. 2. Diseños/1.0 Patron de diseño/
- BOCATTO V2.0 — Diagramas de clases, componentes y ER. 2. Diseños/1.3 Diagramas de Clase/, 2. Diseños/1.4 Diagramas de componentes/, 2. Diseños/1.5 Diagramas ER/
- BOCATTO V2.0 — Guías de la asignatura Análisis y Diseño de Software. 0. Analisis/G2_30723_GUIA{1,2}_ADS_V{1..4}.pdf
- ECMAScript 2022 (ES13) — Language Specification. <https://tc39.es/ecma262/>
- Jest — JavaScript testing framework (backend). <https://jestjs.io/docs/getting-started>
- Vitest — Testing framework con soporte nativo para ESM y jsdom (frontend). <https://vitest.dev/guide/>
- jsdom — Implementación en JavaScript del DOM para Node.js. <https://github.com/jsdom/jsdom>

3. Elementos a Probar

Los siguientes elementos están en el alcance de las pruebas unitarias.

3.1 Elementos del backend (módulos Node.js, ES modules)

Cada elemento es un *único archivo fuente* salvo que se indique lo contrario. Las importaciones/exportaciones se siguen solo cuando es necesario para instanciar la unidad. Rutas relativas a 3. Código/BOCATTO_V3.0/.

<i>ID</i>	<i>Elemento</i>	<i>Ruta</i>
BE-01	Clase ApiError	backend/src/utls/ApiError.js
BE-02	Clase ApiResponse	backend/src/utls/ApiResponse.js
BE-03	UsuarioValidator	backend/src/validators/UsuarioValidator.js
BE-04	ClienteValidator	backend/src/validators/ClienteValidator.js
BE-05	MesaValidator	backend/src/validators/MesaValidator.js
BE-06	MenuValidator	backend/src/validators/MenuValidator.js
BE-07	PedidoValidator	backend/src/validators/PedidoValidator.js
BE-08	PromocionValidator	backend/src/validators/PromocionValidator.js
BE-09	MesaDTO	backend/src/dto/MesaDTO.js
BE-10	MesaMapper	backend/src/mappers/MesaMapper.js
BE-11	PromocionPrototype	backend/src/patterns/prototype/PromocionPrototype.js
BE-12	MenuPrototype	backend/src/patterns/prototype/MenuPrototype.js
BE-13	Prototype (abstracta)	backend/src/patterns/prototype/Prototype.js
BE-14	PedidoBuilder	backend/src/patterns/builder/PedidoBuilder.js
BE-15	DirectorPedido	backend/src/patterns/builder/DirectorPedido.js
BE-16	MesaService	backend/src/services/MesaService.js (repositorio mockeado)
BE-17	AuthService	backend/src/services/AuthService.js (repositorio mockeado)
BE-18	Fábrica roleMiddleware	backend/src/middlewares/roleMiddleware.js
BE-19	authMiddleware	backend/src/middlewares/authMiddleware.js
BE-20	errorMiddleware	backend/src/middlewares/errorMiddleware.js

3.2 Elementos del frontend (Vanilla JS, ES modules)

Rutas relativas a 3. Código/BOCATTO_V3.0/.

ID	Elemento	Ruta
FE-01	Singleton Storage	frontend/utils/storage.js
FE-02	Singleton Auth	frontend/utils/Auth.js
FE-03	Constantes EstadoPedido	frontend/utils/EstadoPedido.js
FE-04	MesaValidator	frontend/validators/MesaValidator.js
FE-05	MenuValidator	frontend/validators/MenuValidator.js
FE-06	PedidoValidator	frontend/validators/PedidoValidator.js
FE-07	ClienteValidator	frontend/validators/ClienteValidator.js
FE-08	UsuarioValidator	frontend/validators/UsuarioValidator.js
FE-09	MesaService (HTTP)	frontend/services/mesaService.js (con stub de fetch)
FE-10	Constante api	frontend/services/api.js
FE-11	Helper Toast	frontend/utils/Toast.js
FE-12	Helper Modal	frontend/utils/Modal.js
FE-13	Helper EventBus	frontend/utils/EventBus.js

Todas las clases de servicio dependientes de HTTP (FE-09 y el resto de archivos *Service.js) se prueban unitariamente **sustituyendo fetch**. La red real no se ejerce en este plan.

3.3 Elementos excluidos de las pruebas unitarias (fuera de alcance)

- Implementaciones de repositorios contra MongoDB (pruebas de integración).
- Orquestación HTTP de controladores contra una aplicación Express en ejecución (pruebas de integración).
- Validación y persistencia del esquema Mongoose (prueba de modelo — integración).
- Flujos end-to-end de Socket.IO (pruebas de sistema).
- Flujo end-to-end de pedidos por QR (pruebas de sistema).
- Seeders (herramientas operativas, no son código del SUT).
- Renderizado HTML, CSS y maquetación visual (pruebas de sistema / aceptación).
- Rendimiento, carga y pruebas de seguridad (planes separados).

4. Características a Probar

Las características siguientes se ejercitarán a nivel unitario. Cada una se mapea a uno o más elementos de la Sección 3.

4.1 Reglas de validación (BE y FE)

<i>ID Caract.</i>	<i>Característica</i>	<i>Elementos ejercitados</i>
F-VAL-01	Rechazar nombre vacío o de solo espacios en menú, cliente, promoción, usuario	BE-03..08, FE-04..08
F-VAL-02	Rechazar precio no positivo en menú	BE-06, FE-05
F-VAL-03	Rechazar cantidad no positiva por ítem en pedido	BE-07, BE-14
F-VAL-04	Requerir mesa para canales distintos de TELEFONO	BE-07, BE-14
F-VAL-05	Requerir clienteNombre, telefonoEntrega y direccionEntrega para el canal TELEFONO	BE-07, BE-14, FE-06
F-VAL-06	Requerir clienteNombre para el canal QR	BE-07
F-VAL-07	Forzar el rango de descuento 0..100 para promociones	BE-08
F-VAL-08	Forzar roles válidos (ADMIN, MESERO, COCINA, CAJA) para usuarios	BE-03
F-VAL-09	Forzar longitud mínima de contraseña (≥6) para usuarios	BE-03
F-VAL-10	Validar el formato de correo electrónico cuando se proporciona (cliente)	BE-04
F-VAL-11	Forzar numero y capacidad > 0 para mesas	BE-05, FE-04

4.2 Lógica de dominio — patrón Builder y Director (BE)

<i>ID Caract.</i>	<i>Característica</i>	<i>Elementos ejercitados</i>
F-BLD-01	El builder reinicia su estado interno en reset()	BE-14
F-BLD-02	El builder asume canal por defecto MESA y convierte a mayúsculas	BE-14
F-BLD-03	El builder rechaza ítems con cantidad <= 0	BE-14
F-BLD-04	El builder rechaza ítems con producto.precio no positivo	BE-14
F-BLD-05	El builder acumula total e ítems	BE-14
F-BLD-06	El builder aplica descuento porcentual mediante aplicarPromocion	BE-14
F-BLD-07	El builder valida los campos requeridos según el canal en validar()	BE-14
F-BLD-08	El builder devuelve una copia profunda en build() y reinicia el estado	BE-14
F-BLD-09	El director orquesta la construcción completa de un pedido (incluye setClienteNombre y setObservaciones)	BE-15

4.3 Patrón Prototype (BE)

<i>ID Caract.</i>	<i>Característica</i>	<i>Elementos ejercitados</i>
F-PRT-01	Prototype.clone() lanza error cuando no se implementa	BE-13

ID Caract.	Característica	Elementos ejercitados
F-PRT-02	PromocionPrototype.clone() devuelve una copia profunda desacoplada	BE-11
F-PRT-03	MenuPrototype.clone() devuelve una copia profunda desacoplada	BE-12

4.4 DTO / Mapper (BE)

ID Caract.	Característica	Elementos ejercitados
F-DTO-01	MesaDTO expone solo la forma pública (id, numero, capacidad, estado)	BE-09
F-MAP-01	MesaMapper.toDTO envuelve una sola entidad	BE-10
F-MAP-02	MesaMapper.toDTOList envuelve un arreglo de entidades	BE-10
F-MAP-03	El mapper maneja un arreglo vacío	BE-10

4.5 Reglas de negocio de la capa de servicio (BE)

ID Caract.	Característica	Elementos ejercitados
F-SRV-01	MesaService.obtenerMesaPorId lanza 404 cuando no existe	BE-16
F-SRV-02	MesaService.cambiarEstado lanza 404 cuando no existe	BE-16
F-SRV-03	MesaService.crearMesa rechaza número duplicado	BE-16
F-SRV-04	MesaService.actualizarMesa rechaza número duplicado (que no sea el mismo registro)	BE-16
F-SRV-05	MesaService.eliminarMesa lanza 404 cuando no existe	BE-16
F-SRV-06	AuthService.login rechaza un username desconocido	BE-17
F-SRV-07	AuthService.login rechaza (403 BLOQUEADO:N) una cuenta bloqueada dentro de la ventana	BE-17
F-SRV-08	AuthService.login auto-desbloquea y continúa si la ventana de bloqueo expiró	BE-17
F-SRV-09	AuthService.login responde 401 INTENTOS:N ante contraseña incorrecta (intentos 1 y 2)	BE-17
F-SRV-10	AuthService.login bloquea la cuenta (403 BLOQUEADO:300) en el 3er intento fallido	BE-17
F-SRV-11	AuthService.login reinicia el estado con desbloquearUsuario y devuelve JWT en caso de éxito	BE-17

4.6 Middleware (BE)

ID Caract.	Característica	Elementos ejercitados
F-MID-01	roleMiddleware retorna 401 cuando falta req.usuario	BE-18
F-MID-02	roleMiddleware retorna 403 cuando el rol no está en la lista permitida	BE-18

ID Caract.	Característica	Elementos ejercitados
F-MID-03	roleMiddleware llama a next() cuando el rol es permitido	BE-18
F-MID-04	authMiddleware rechaza solicitudes sin encabezado Authorization o con uno inválido	BE-19
F-MID-05	authMiddleware carga req.usuario a partir de un JWT válido	BE-19
F-MID-06	errorMiddleware renderiza el sobre estándar de error	BE-20

4.7 Utilidad HTTP / respuesta (BE)

ID Caract.	Característica	Elementos ejercitados
F-UTL-01	ApiError extiende Error y expone status	BE-01
F-UTL-02	ApiError.status vale 500 por defecto	BE-01
F-UTL-03	ApiResponse.success retorna JSON con success: true, message, data	BE-02
F-UTL-04	ApiResponse.success respeta un código de estado personalizado	BE-02

4.8 Frontend — clases utilitarias (FE)

ID Caract.	Característica	Elementos ejercitados
F-FEU-01	Storage persiste y lee el token	FE-01
F-FEU-02	Storage persiste y lee el usuario	FE-01
F-FEU-03	Storage.limpiarSesion elimina tanto token como usuario	FE-01
F-FEU-04	Storage.estaAutenticado retorna false cuando no hay token	FE-01
F-FEU-05	Auth.autenticado delega en Storage	FE-02
F-FEU-06	Auth.rol retorna el rol del usuario actual	FE-02
F-FEU-07	Auth.tieneRol(...roles) coincide con uno de los roles	FE-02
F-FEU-08	EstadoPedido expone constantes canónicas de estado	FE-03

4.9 Frontend — servicios sobre fetch (FE)

ID Caract.	Característica	Elementos ejercitados
F-FES-01	MesaService.listar llama a GET /mesas con token bearer (URL base V3: https://bocatto-2gaz.onrender.com/api/v1)	FE-09
F-FES-02	MesaService.crear llama a POST /mesas con cuerpo serializado	FE-09
F-FES-03	MesaService.actualizar llama a PUT /mesas/:id	FE-09
F-FES-04	MesaService.eliminar llama a DELETE /mesas/:id	FE-09

<i>ID Caract.</i>	<i>Característica</i>	<i>Elementos ejercitados</i>
F-FES-05	MesaService.cambiarEstado llama a PATCH /mesas/:id/estado	FE-09
F-FES-06	El encabezado Authorization se omite cuando no existe sesión (corrección V3.0)	FE-09

4.10 Frontend — bus de eventos y helpers de UI (FE)

<i>ID Caract.</i>	<i>Característica</i>	<i>Elementos ejercitados</i>
F-FEE-01	EventBus.emitir/escuchar/eliminar despacha eventos vía document.dispatchEvent+CustomEvent	FE-13
F-FEE-02	EventBus.eliminar quita un listener específico	FE-13
F-FET-01	Toast.success/error/warning/info renderiza un elemento con el nivel solicitado (requiere bootstrap global)	FE-11
F-FEM-01	Modal.abrir/cerrar alterna el elemento modal vía bootstrap.Modal	FE-12

5. Características que No se Probarán

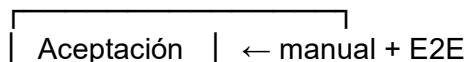
Las siguientes se excluyen explícitamente de este **plan de pruebas unitarias** y se cubren en otros niveles de prueba:

1. **Integración con MongoDB** — Repositorios, modelos de Mongoose, persistencia: cubierto por un plan de pruebas de integración.
2. **Enrutamiento y orquestación HTTP** — Controladores de Express, flujo de petición/respuesta: plan de pruebas de integración / API.
3. **Flujos end-to-end** — Inicio de sesión, colocación completa de un pedido, de cocina a entrega, cierre de caja: plan de pruebas de sistema.
4. **Canales en tiempo real de Socket.IO** — cubiertos por el plan de pruebas de sistema.
5. **Flujo público por QR** — cubierto por el plan de pruebas de sistema.
6. **Requisitos no funcionales** — rendimiento, carga, escalabilidad, accesibilidad, pruebas de penetración de seguridad: cubiertos por planes dedicados.
7. **Seeders y scripts de migración de base de datos** — herramientas operativas, no son código del SUT.
8. **Renderizado HTML y maquetación visual** — cubiertos por el plan de pruebas de aceptación.

6. Enfoque de Pruebas

6.1 Niveles de prueba

Este plan cubre únicamente **pruebas unitarias** (el nivel más bajo). La pirámide de pruebas del proyecto es:



Sistema	← E2E / Cypress o equivalente
Integración	← supertest + mongodb-memory-server
UNITARIA	← este plan (Jest / Vitest)

6.2 Técnica de prueba

Las técnicas de prueba utilizadas son:

Código	Técnica	Usada para
T1	Partición de equivalencia	reglas de validación, reglas de negocio
T2	Análisis de valores límite	límites numéricos (cantidad, capacidad, descuento, precio)
T3	Tabla de decisión	validación guiada por canal en PedidoValidator / PedidoBuilder
T4	Transición de estados	máquina de estados de bloqueo en AuthService.login
T5	Adivinación de errores	nulo, indefinido, cadena vacía, tipos incorrectos, entradas enormes
T6	Aislamiento basado en mocks	AuthService, MesaService, MesaService (HTTP)

6.3 Framework de pruebas

Capa	Framework	Runner	Librería de aserciones	Mocking
Backend	Jest	jest	Jest expect	Mocks de Jest / jest.mock()
Frontend	Vitest + jsdom	vitest	Vitest expect	vi.fn(), vi.stubGlobal

Se recomienda Vitest para el frontend porque consume la estructura actual de ES modules sin transpilación y soporta jsdom para window / localStorage.

6.4 Convención de estructura de pruebas

backend/

src/

utils/

ApiError.js

ApiError.spec.js ← spec unitaria colocalizada

validators/

MesaValidator.js

MesaValidator.spec.js

...

jest.config.cjs

package.json (añade scripts: "test", "test:unit", "test:coverage")

frontend/

utils/

storage.js

storage.spec.js

validators/

MesaValidator.spec.js

services/

```

    mesaService.spec.js
    vitest.config.js
    package.json (añade scripts: "test", "test:unit", "test:coverage")

```

6.5 Patrón de diseño de pruebas

Todas las pruebas unitarias siguen el patrón **Arrange / Act / Assert (AAA)**:

```

test("MesaValidator rechaza capacidad negativa", () => {
  // Arrange
  const datos = { numero: 5, capacidad: -1 };

  // Act + Assert
  expect(() => MesaValidator.validar(datos)).toThrow(ApiError);
});

```

6.6 Estrategia de mocks

- **Servicios del backend:** el repositorio y bcrypt / jwt se sustituyen con mocks de Jest. La unidad bajo prueba es el servicio en sí.
- **Middlewares del backend:** las pruebas reciben objetos req / res simulados y un espía next.
- **Servicios del frontend:** global.fetch se reemplaza con vi.stubGlobal y manejadores vi.fn() por prueba.
- **Almacenamiento del frontend:** el localStorage de jsdom se reinicia entre pruebas.
- **No se usan red real ni base de datos real.**

6.7 Objetivos de cobertura de código

Capa	Objetivo
Utilidades backend	≥ 95% cobertura de líneas
Validadores backend	100% cobertura de líneas
Patrones backend	100% cobertura de líneas
Servicios backend	≥ 85% cobertura de líneas
Middleware backend	≥ 90% cobertura de líneas
Validadores frontend	100% cobertura de líneas
Utilidades frontend	≥ 90% cobertura de líneas
Servicios frontend	≥ 80% cobertura de líneas
Backend global	≥ 90% cobertura de líneas
Frontend global	≥ 85% cobertura de líneas

Una cobertura por debajo del objetivo requiere justificación explícita en el informe resumen de pruebas.

6.8 Gestión de datos de prueba

- Los fixtures de prueba se almacenan en carpetas `__fixtures__` / colocadas o se construyen inline en el spec.
- No se usan datos de producción. No se cargan seeders.
- Cada prueba reinicia sus mocks y el DOM en memoria.

6.9 Herramientas e Integración Continua

- Linting: **ESLint** con las reglas existentes del proyecto.

- Flujo de CI: lint → test:backend → test:frontend en cada Pull Request.
- El reporte de cobertura se publica como artefacto del CI.

7. Criterios de Aprobación/Fallo

7.1 Aprobación/Fallo por caso de prueba

Un caso de prueba **pasa** cuando:

1. El resultado real coincide exactamente con el resultado esperado (o con un matcher explícito, p. ej. `toThrow(ApiError)`).
2. No se lanza ninguna excepción no manejada distinta a la afirmada.
3. Todas las aserciones dentro del cuerpo de la prueba se satisfacen.

Un caso de prueba **falla** cuando se incumple cualquiera de las condiciones anteriores.

7.2 Aprobación/Fallo por módulo

Un módulo **pasa** cuando:

- El 100% de sus casos de prueba pasan.
- La cobertura alcanza el objetivo por capa definido en §6.7.

Un módulo **falla** cuando cualquier caso de prueba falla o la cobertura cae por debajo del objetivo.

7.3 Aprobación/Fallo por componente (backend / frontend)

Un componente **pasa** cuando todos los módulos en alcance de dicho componente pasan.

7.4 Aprobación/Fallo global

La fase de pruebas unitarias se considera **completa y aprobada** cuando:

- Cada elemento en alcance de §3 tiene al menos un caso de prueba aprobado.
- Se cumplen los objetivos de cobertura de §6.7 o se renuncian formalmente.
- No hay defectos abiertos de severidad 1 (ver §13.2).

8. Criterios de Suspensión y Requisitos de Reanudación

8.1 Criterios de suspensión (cuándo detener la fase)

La fase de pruebas unitarias se **suspenderá** si ocurre cualquiera de los siguientes eventos:

Código	Disparador
S-01	La compilación/carga del build bajo prueba falla (p. ej. errores de import en ESM).

Código	Disparador
S-02	Más del 20% de los casos de prueba de un mismo módulo fallan en la primera ejecución.
S-03	La cobertura del backend o frontend cae por debajo del 60% en un gate temprano.
S-04	Se descubre un defecto de severidad 1 en el código de producción que bloquea más de 5 tests.
S-05	Una dependencia externa crítica no está disponible (runner de CI, registro de npm).
S-06	El código bajo prueba se modifica mientras un ciclo de pruebas está en curso.

8.2 Requisitos de reanudación

Las pruebas solo se reanudarán cuando:

Código	Requisito
R-01	La compilación es limpia y todos los archivos spec cargan sin errores.
R-02	Existe una corrección documentada para cada defecto bloqueante, o el defecto fue diferido por el Líder de QA.
R-03	Se añade un plan de re-prueba al calendario (Sección 12).
R-04	Se reevalúa la línea base de cobertura tras la corrección.

9. Entregables de las Pruebas

Entregable	Descripción
Plan de pruebas (este documento)	Plan de pruebas unitarias según IEEE 829
Especificaciones de diseño de prueba	Por cada elemento: entradas, salidas, necesidades del entorno (Apéndices A y B)
Especificaciones de casos de prueba	Catalogados en el Apéndice C
Especificaciones de procedimientos de prueba	Archivos *.spec.js — especificaciones ejecutables (Jest / Vitest)
Bitácoras de prueba	Salida del pipeline de CI + bitácoras por módulo
Reportes de anomalías de prueba	Por cada test fallido: id, resumen, entradas, esperado, real, severidad, entorno
Informe resumen de pruebas	Al cierre de la fase: conteos pasa/falla, cobertura, defectos, sign-off
Reporte de cobertura	HTML + Icov, publicado por el CI

Convención de nombres: TP-BOCATTO-UT-001 para este plan, TC-BOCATTO-UT-NNN para casos de prueba, TA-BOCATTO-UT-NNN para reportes de anomalía.

10. Tareas de Pruebas

ID	Tarea	Responsable	Depende de
T-01	Añadir Jest al backend, configurar ESM y entorno Node sin jsdom	Backend	—
T-02	Añadir Vitest + jsdom al frontend, configurar scripts	Frontend	—
T-03	Crear archivos spec para utilerías del backend (ApiError, ApiResponse)	Backend	T-01

<i>ID</i>	<i>Tarea</i>	<i>Responsabl e</i>	<i>Depende de</i>
T-04	Crear archivos spec para validadores del backend	Backend	T-01
T-05	Crear archivos spec para patrones del backend (Builder, Director, Prototype)	Backend	T-01
T-06	Crear archivos spec para DTOs y mappers del backend	Backend	T-01
T-07	Crear archivos spec para servicios del backend (repositorios mockeados)	Backend	T-01
T-08	Crear archivos spec para middlewares del backend	Backend	T-01
T-09	Crear archivos spec para utilerías del frontend (Storage, Auth, EstadoPedido)	Frontend	T-02
T-10	Crear archivos spec para validadores del frontend	Frontend	T-02
T-11	Crear archivos spec para servicios del frontend (fetch sustituido)	Frontend	T-02
T-12	Crear archivos spec para helpers de UI del frontend (Toast, Modal, EventBus)	Frontend	T-02
T-13	Cablear el flujo de CI: lint, test, cobertura	DevOps	T-01, T-02
T-14	Generar el reporte de cobertura y validar objetivos	Líder de QA	T-13
T-15	Producir el informe resumen de pruebas	Líder de QA	T-14

11. Necesidades del Entorno

11.1 Hardware

- Equipos de desarrollo: 1 CPU, 4 GB de RAM mínimo.
- Runner de CI: 2 CPU, 4 GB de RAM (GitHub Actions ubuntu-latest o equivalente).

11.2 Software

<i>Elemento</i>	<i>Backend</i>	<i>Frontend</i>
<i>Runtime</i>	Node.js ≥ 20 LTS	Node.js ≥ 20 LTS (solo para el runner)
<i>Framework de pruebas</i>	Jest 29	Vitest 1.x
<i>Emulación de DOM</i>	no requerida	jsdom
<i>Linting</i>	ESLint (existente)	ESLint (existente)
<i>Mocking</i>	Mocks de Jest	vi.fn, vi.stubGlobal
<i>Reporte de cobertura</i>	c8 o Jest --coverage	@vitest/coverage-v8
<i>CI</i>	GitHub Actions	GitHub Actions

11.3 Red

- Las pruebas **no deben** requerir acceso a la red. fetch se sustituye; no hay llamadas a la API real.
- El registro de npm solo se permite en el momento de la instalación.

11.4 Archivos de configuración a añadir

- backend/jest.config.cjs — testEnvironment: "node", consciente de ESM, transform: {}.

- backend/package.json — "test": "jest", "test:coverage": "jest --coverage".
- frontend/vitest.config.js — environment: "jsdom".
- frontend/package.json — "test": "vitest run", "test:coverage": "vitest run --coverage".

12. Calendario

El cronograma se expresa en días hábiles desde el inicio de la fase de pruebas unitarias (T0).

<i>Fase</i>	<i>Días</i>	<i>Criterio de salida</i>
<i>K0 — Preparación del entorno de pruebas (T-01, T-02)</i>	1	Todos los comandos de prueba corren con 0 tests
<i>K1 — Utilidades y validadores del backend (T-03, T-04)</i>	2	Todos los spec BE-01..08 en verde
<i>K2 — Patrones del backend (T-05)</i>	1	BE-11..15 en verde
<i>K3 — DTO/mapper del backend (T-06)</i>	1	BE-09, BE-10 en verde
<i>K4 — Servicios y middleware del backend (T-07, T-08)</i>	3	BE-16..20 en verde
<i>K5 — Utilerías y validadores del frontend (T-09, T-10)</i>	2	FE-01..08 en verde
<i>K6 — Servicios y helpers de UI del frontend (T-11, T-12)</i>	2	FE-09..13 en verde
<i>K7 — Integración con CI (T-13)</i>	1	Flujo de CI en verde
<i>K8 — Validación de cobertura y sign-off (T-14, T-15)</i>	1	Informe resumen de pruebas aprobado
Total	14	

Hitos:

- **M1** fin de K0 — entorno listo.
- **M2** fin de K4 — pruebas unitarias del backend completas.
- **M3** fin de K6 — pruebas unitarias del frontend completas.
- **M4** fin de K8 — fase de pruebas unitarias firmada.

13. Riesgos y Contingencias

13.1 Riesgos del proyecto

<i>ID</i>	<i>Riesgo</i>	<i>Pro babi lida d</i>	<i>I m p a c t o</i>	<i>Mitigación</i>
R-1	El acoplamiento fuerte de los servicios a los repositorios dificulta el mocking	Medi a	Medio	Usar fábricas de mocks de Jest; refactorizar servicios para inyectar repositorios (sin cambio de alcance)
R-2	La interoperabilidad ESM + Jest retrasa la configuración	Medi a	Bajo	Usar el soporte experimental ESM de Jest o babel-jest solo si es estrictamente necesario
R-3	Los servicios del frontend no están diseñados para ser testeables (sin DI)	Alta	Medio	Sustituir global.fetch; tratar los servicios como caja negra por método
R-4	El comportamiento de los validadores diverge entre frontend y backend	Alta	Medio	Crear pruebas espejo; documentar las diferencias intencionales
R-5	Los objetivos de cobertura se incumplen por presión de tiempo	Medi a	Medio	Gate de cobertura en CI; las desviaciones requieren sign-off formal en el resumen
R-6	Cambios continuos en el código fuente durante la fase de pruebas	Medi a	Alto	Aplicar los criterios de suspensión de §8; fijar una etiqueta al inicio de la fase
R-7	Falta de tiempo o personal para crear todos los archivos spec	Medi a	Alto	Priorizar validadores y patrones; diferir los helpers de UI a una ola posterior

13.2 Esquema de severidad de defectos

<i>Severid ad</i>	<i>Definición</i>
S1	Bloquea la fase de pruebas; el test no puede ejecutarse o revela lógica de negocio central incorrecta.
S2	El test falla; el defecto afecta una característica documentada, pero no bloquea la fase.
S3	Cosmético/menor; el test pasa con un workaround documentado.
S4	Solo documentación/nomenclatura/estilo.

14. Aprobaciones

<i>Rol</i>	<i>Nombre</i>	<i>Firma</i>	<i>Fecha</i>
Líder de QA			
Tech Lead BE			

Rol	Nombre	Firma	Fecha
Tech Lead FE			
Project Manager			

Apéndice A — Especificaciones de Pruebas Unitarias del Backend

Este apéndice aplica la plantilla de especificación de diseño de prueba de IEEE 829 a cada elemento bajo prueba del backend.

A.1 BE-01 — ApiError (backend/src/utils/ApiError.js)

- **Característica(s) probada(s):** F-UTL-01, F-UTL-02
- **Enfoque:** Caja negra (T5 — adivinación de errores). Se instancia la clase con argumentos y se inspeccionan propiedades.
- **Criterio de aprobación:** El objeto es instanceof Error e instanceof ApiError; status refleja la entrada; status vale 500 por defecto.
- **Esquema del spec (Jest):**

```
import ApiError from "../ApiError.js";
```

```
describe("ApiError", () => {
  it("extiende Error y expone status 500 por defecto", () => {
    const e = new ApiError("boom");
    expect(e).toBeInstanceOf(Error);
    expect(e).toBeInstanceOf(ApiError);
    expect(e.message).toBe("boom");
    expect(e.status).toBe(500);
  });

  it("usa el código de estado proporcionado", () => {
    const e = new ApiError("bad", 400);
    expect(e.status).toBe(400);
  });
});
```

A.2 BE-02 — ApiResponse (backend/src/utils/ApiResponse.js)

- **Característica(s) probada(s):** F-UTL-03, F-UTL-04
- **Enfoque:** Caja negra; se pasa un objeto res simulado que expone status().json().
- **Criterio de aprobación:** El cuerpo JSON contiene success: true, el message proporcionado y data. El código de estado coincide con el parámetro.
- **Esquema del spec:**

```
import ApiResponse from "../ApiResponse.js";
```

```

const fakeRes = () => {
  const r = {};
  r.status = jest.fn().mockReturnValue(r);
  r.json = jest.fn().mockReturnValue(r);
  return r;
};

describe("ApiResponse.success", () => {
  it("retorna el sobre de éxito con 200 por defecto", () => {
    const r = fakeRes();
    ApiResponse.success(r, "ok", { a: 1 });
    expect(r.status).toHaveBeenCalledWith(200);
    expect(r.json).toHaveBeenCalledWith({ success: true, message: "ok", data: { a: 1 } });
  });

  it("respeta un código de estado personalizado", () => {
    const r = fakeRes();
    ApiResponse.success(r, "created", null, 201);
    expect(r.status).toHaveBeenCalledWith(201);
  });
});

```

A.3 BE-03..08 — Validadores

Enfoque común (T1, T2, T5):

- Camino feliz → no lanza.
- Cada regla violada → lanza ApiError con el status esperado (400) y la subcadena de mensaje esperada.
- Valores límite: 0, 1, -1, max, max+1.

Ejemplo para MesaValidator (BE-05):

```

import MesaValidator from "../MesaValidator.js";
import ApiError from "../utils/ApiError.js";

describe("MesaValidator", () => {
  it("acepta una mesa válida", () => {
    expect(() => MesaValidator.validar({ numero: 1, capacidad: 4 })).not.toThrow();
  });

  it("rechaza numero faltante (BVA: vacío/falsy)", () => {
    expect(() => MesaValidator.validar({ capacidad: 4 })).toThrow(ApiError);
  });

  it("rechaza capacidad < 1 (BVA: 0 y negativo)", () => {

```

```

    expect(() => MesaValidator.validar({ numero: 1, capacidad: 0 })).toThrow(/Capacidad
    inválida/);
    expect(() => MesaValidator.validar({ numero: 1, capacidad: -3 })).toThrow(/Capacidad
    inválida/);
  });
});

```

La misma plantilla aplica a:

- UsuarioValidator — campos vacíos, longitud de contraseña 5/6/7, rol válido, rol inválido.
- ClienteValidator — nombre vacío, correo mal formado, correo válido.
- MenuValidator — nombre vacío, precio 0, precio negativo, precio positivo.
- PedidoValidator — prueba basada en tabla (T3) para la tabla de decisión canal × campos requeridos:

<i>canal</i>	<i>m es a</i>	<i>telefonoEntrega</i>	<i>clienteNombre</i>	<i>resultado esperado</i>
MESA	✓	n/a	n/a	ok
MESA	✗	n/a	n/a	lanza "Debe seleccionar una mesa"
TELEFONO	n/a	✓	n/a	ok
TELEFONO	n/a	✗	n/a	lanza "teléfono"
QR	n/a	n/a	✓	ok
QR	n/a	n/a	✗	lanza "QR requiere nombre"
(por defecto)	✗	n/a	n/a	lanza "mesa"

Más los casos de items con longitud 0 y cantidad <= 0.

- PromocionValidator — nombre vacío, descuento 0, 100, -1, 101.

A.4 BE-09..10 — MesaDTO y MesaMapper

Enfoque (T1): construir DTO/Mapper con entidades fabricadas a mano, asegurar la forma de salida.

```

import MesaDTO from "./MesaDTO.js";
import MesaMapper from "./MesaMapper.js";

describe("MesaDTO", () => {
  it("mapea una entidad tipo Mongoose a su forma pública", () => {
    const d = new MesaDTO({ _id: "x1", numero: 7, capacidad: 4, estado: "LIBRE", extra:
    "ignored" });
    expect(d).toEqual({ id: "x1", numero: 7, capacidad: 4, estado: "LIBRE" });
  });
});

describe("MesaMapper", () => {
  it("envuelve un listado de mesas", () => {
    const list = MesaMapper.toDTOList([

```

```

    { _id: "a", numero: 1, capacidad: 2, estado: "LIBRE" },
    { _id: "b", numero: 2, capacidad: 4, estado: "OCUPADA" },
  ]);
  expect(list).toHaveLength(2);
  expect(list[0]).toBeInstanceOf(MesaDTO);
});

it("maneja un arreglo vacío", () => {
  expect(MesaMapper.toDTOList([])).toEqual([]);
});
});

```

A.5 BE-11..13 — Patrón Prototype

```

import Prototype from "../Prototype.js";
import PromocionPrototype from "../PromocionPrototype.js";
import MenuPrototype from "../MenuPrototype.js";

describe("Prototype (abstracta)", () => {
  it("lanza error cuando clone() no está implementado", () => {
    const p = new Prototype();
    expect(() => p.clone()).toThrow(/clone\(\)/);
  });
});

describe("PromocionPrototype", () => {
  it("devuelve una copia profunda", () => {
    const src = { nombre: "X", descuento: 10, nested: { a: 1 } };
    const clone = new PromocionPrototype(src).clone();
    expect(clone).toEqual(src);
    expect(clone).not.toBe(src);
    expect(clone.nested).not.toBe(src.nested);
  });
});

describe("MenuPrototype", () => {
  it("devuelve una copia profunda", () => {
    const src = { nombre: "M", precio: 1.5, tags: ["a", "b"] };
    const clone = new MenuPrototype(src).clone();
    expect(clone).toEqual(src);
    expect(clone).not.toBe(src);
    expect(clone.tags).not.toBe(src.tags);
  });
});

```

A.6 BE-14..15 — PedidoBuilder y DirectorPedido

Enfoque (T1, T2, T3, T5): ejercitar cada setter, cada regla de validación y la orquestación a través del director.

```
import PedidoBuilder from "../PedidoBuilder.js";
import DirectorPedido from "../DirectorPedido.js";
import ApiError from "../../utils/ApiError.js";

const prod = (over = {}) => ({ _id: "p1", nombre: "Café", precio: 2.5, ...over });

describe("PedidoBuilder", () => {
  it("se reinicia a un pedido MESA con estado PENDIENTE y total 0", () => {
    const b = new PedidoBuilder();
    b.setMesa("M1").agregarProducto(prod(), 1);
    expect(b.pedido.items).toHaveLength(1);
    b.reset();
    expect(b.pedido).toMatchObject({ canal: "MESA", mesa: null, items: [], total: 0, estado:
"PENDIENTE" });
  });

  it("convierte canal a mayúsculas y por defecto es MESA", () => {
    const b = new PedidoBuilder();
    b.setCanal("telefono");
    expect(b.pedido.canal).toBe("TELEFONO");
    b.reset();
    expect(b.pedido.canal).toBe("MESA");
  });

  it("rechaza cantidad <= 0", () => {
    const b = new PedidoBuilder();
    expect(() => b.agregarProducto(prod(), 0)).toThrow(ApiError);
  });

  it("rechaza producto sin precio positivo", () => {
    const b = new PedidoBuilder();
    expect(() => b.agregarProducto({ ...prod(), precio: 0 }, 1)).toThrow(/Precio inválido/);
  });

  it("acumula total e items", () => {
    const b = new PedidoBuilder();
    b.agregarProducto(prod(), 2).agregarProducto(prod({ precio: 3 }), 1);
    expect(b.pedido.items).toHaveLength(2);
    expect(b.pedido.total).toBeCloseTo(2.5 * 2 + 3, 2);
  });
});
```

```

it("aplica un descuento porcentual", () => {
  const b = new PedidoBuilder();
  b.agregarProducto(prod(), 2).aplicarPromocion({ descuento: 50 });
  expect(b.pedido.total).toBeCloseTo(2.5);
});

it("requiere mesa para canales distintos de TELEFONO", () => {
  const b = new PedidoBuilder();
  b.agregarProducto(prod(), 1);
  expect(() => b.validar()).toThrow(/mesa/i);
});

it("requiere teléfono para el canal TELEFONO", () => {
  const b = new PedidoBuilder();
  b.setCanal("TELEFONO").agregarProducto(prod(), 1);
  expect(() => b.validar()).toThrow(/teléfono/i);
});

it("expone setClienteNombre y setObservaciones", () => {
  const b = new PedidoBuilder();
  b.setClienteNombre("Juan").setObservaciones("sin sal");
  expect(b.pedido.clienteNombre).toBe("Juan");
  expect(b.pedido.observaciones).toBe("sin sal");
});

it("build() retorna una copia profunda y reinicia el estado", () => {
  const b = new PedidoBuilder();
  b.setMesa("M1").agregarProducto(prod(), 1);
  const out = b.build();
  expect(out).toMatchObject({ canal: "MESA", mesa: "M1" });
  expect(b.pedido.items).toHaveLength(0);
});

describe("DirectorPedido", () => {
  it("construye un pedido completo a partir de datos crudos (incluye clienteNombre y observaciones)", () => {
    const out = DirectorPedido.construirPedido({
      canal: "MESA",
      mesa: "M9",
      cliente: null,
      clienteNombre: "Maria",
      mesero: "u1",
      observaciones: "llevar",
    });
  });
});

```



```

    items: [{ producto: prod(), cantidad: 3 }],
  });
  expect(out).toMatchObject({ canal: "MESA", mesa: "M9", total: 7.5, clienteNombre: "Maria",
observaciones: "llevar" });
  expect(out.items[0].subtotal).toBeCloseTo(7.5);
});

it("aplica un descuento cuando se proporciona una promoción", () => {
  const out = DirectorPedido.construirPedido({
    canal: "MESA",
    mesa: "M9",
    items: [{ producto: prod(), cantidad: 2 }],
    promocion: { descuento: 25 },
  });
  expect(out.total).toBeCloseTo(2.5 * 2 * 0.75);
});
});

```

A.7 BE-16 — MesaService

Enfoque (T1, T5): se mockea el repositorio. Se verifican las ramas.

```

jest.mock("../repositories/MesaRepository.js");
import MesaRepository from "../repositories/MesaRepository.js";
import MesaService from "../MesaService.js";
import ApiError from "../utils/ApiError.js";

describe("MesaService", () => {
  it("listarMesas delega en el repositorio", async () => {
    MesaRepository.obtenerTodas.mockResolvedValue([{ _id: "m1" }]);
    await expect(MesaService.listarMesas()).resolves.toEqual([{ _id: "m1" }]);
  });

  it("obtenerMesaPorId lanza 404 cuando no existe", async () => {
    MesaRepository.obtenerPorId.mockResolvedValue(null);
    await expect(MesaService.obtenerMesaPorId("x")).rejects.toMatchObject({ status: 404 });
  });

  it("crearMesa rechaza numero duplicado", async () => {
    MesaRepository.obtenerPorNumero.mockResolvedValue({ _id: "m1", numero: 3 });
    await expect(MesaService.crearMesa({ numero: 3 })).rejects.toMatchObject({ status: 400 });
  });

  it("actualizarMesa rechaza numero duplicado de otra mesa", async () => {
    MesaRepository.obtenerPorNumero.mockResolvedValue({ _id: "other", numero: 3 });
    await expect(MesaService.actualizarMesa("m1", { numero: 3 })).rejects.toMatchObject({

```

```
status: 400 });  
});
```

```
it("actualizarMesa permite conservar el mismo numero", async () => {  
  MesaRepository.obtenerPorNumero.mockResolvedValue({ _id: "m1", numero: 3 });  
  MesaRepository.actualizar.mockResolvedValue({ _id: "m1", numero: 3 });  
  const r = await MesaService.actualizarMesa("m1", { numero: 3 });  
  expect(r).toMatchObject({ _id: "m1" });  
});
```

```
it("eliminarMesa lanza 404 cuando no existe", async () => {  
  MesaRepository.eliminar.mockResolvedValue(null);  
  await expect(MesaService.eliminarMesa("x")).rejects.toMatchObject({ status: 404 });  
});  
});
```

A.8 BE-17 — AuthService

Enfoque (T4 — máquina de estados de bloqueo temporal, T6 — mocking). V3.0 introdujo bloqueadoHasta y desbloquearUsuario; los fallos de contraseña devuelven ApiError con INTENTOS:N (401) o BLOQUEADO:300 (403).

```
jest.mock("bcryptjs");  
jest.mock("jsonwebtoken");  
jest.mock("../repositories/UsuarioRepository.js");
```

```
import bcrypt from "bcryptjs";  
import jwt from "jsonwebtoken";  
import UsuarioRepository from "../repositories/UsuarioRepository.js";  
import AuthService from "../AuthService.js";
```

```
const user = (over = {}) => ({  
  _id: "u1", username: "admin", password: "h", bloqueado: false,  
  intentosFallidos: 0, rol: "ADMIN", bloqueadoHasta: null, ...over,  
});
```

```
describe("AuthService.login", () => {  
  beforeEach(() => {  
    jest.clearAllMocks();  
    process.env.JWT_SECRET = "secret";  
  });
```

```
it("rechaza username desconocido", async () => {  
  UsuarioRepository.obtenerPorUsername.mockResolvedValue(null);  
  await expect(AuthService.login("x", "y")).rejects.toThrow(/no existe/);  
});
```

```

it("rechaza (403 BLOQUEADO:N) una cuenta bloqueada dentro de la ventana", async () => {
  const futuro = new Date(Date.now() + 120_000);
  UsuarioRepository.obtenerPorUsername.mockResolvedValue(user({ bloqueado: true,
bloqueadoHasta: futuro }));
  await expect(AuthService.login("admin", "x")).rejects.toMatchObject({ status: 403 });
});

it("auto-desbloquea y continúa si la ventana de bloqueo expiró", async () => {
  const pasado = new Date(Date.now() - 60_000);
  UsuarioRepository.obtenerPorUsername.mockResolvedValue(user({ bloqueado: true,
bloqueadoHasta: pasado }));
  UsuarioRepository.desbloquearUsuario.mockResolvedValue();
  bcrypt.compare.mockResolvedValue(true);
  jwt.sign.mockReturnValue("tok");
  const r = await AuthService.login("admin", "ok");
  expect(UsuarioRepository.desbloquearUsuario).toHaveBeenCalledWith("u1");
  expect(r.token).toBe("tok");
});

it("responde 401 INTENTOS:N ante contraseña incorrecta (intentos 1 y 2)", async () => {
  UsuarioRepository.obtenerPorUsername.mockResolvedValue(user());
  bcrypt.compare.mockResolvedValue(false);
  UsuarioRepository.actualizarIntentos.mockResolvedValue();
  await expect(AuthService.login("admin", "bad")).rejects.toMatchObject({ status: 401,
message: /INTENTOS:2/ });
  expect(UsuarioRepository.actualizarIntentos).toHaveBeenCalledWith("u1", 1);
});

it("bloquea la cuenta (403 BLOQUEADO:300) en el 3er intento fallido", async () => {
  UsuarioRepository.obtenerPorUsername.mockResolvedValue(user({ intentosFallidos: 2 }));
  bcrypt.compare.mockResolvedValue(false);
  UsuarioRepository.bloquearUsuario.mockResolvedValue();
  await expect(AuthService.login("admin", "bad")).rejects.toMatchObject({ status: 403,
message: /BLOQUEADO:300/ });
  expect(UsuarioRepository.bloquearUsuario).toHaveBeenCalledWith("u1");
});

it("reinicia el estado con desbloquearUsuario y devuelve JWT en caso de éxito", async () => {
  UsuarioRepository.obtenerPorUsername.mockResolvedValue(user({ intentosFallidos: 2 }));
  bcrypt.compare.mockResolvedValue(true);
  UsuarioRepository.desbloquearUsuario.mockResolvedValue();
  jwt.sign.mockReturnValue("token.x.y");
  const r = await AuthService.login("admin", "ok");
  expect(UsuarioRepository.desbloquearUsuario).toHaveBeenCalledWith("u1");
});

```

```
    expect(r.token).toBe("token.x.y");
    expect(r.usuario.username).toBe("admin");
  });
});
```

A.9 BE-18..20 — Middleware

Enfoque (T1, T5): se proveen req / res simulados y un espía next.

```
import roleMiddleware from "../roleMiddleware.js";
```

```
const fakeRes = () => ({ status: jest.fn().mockReturnThis(), json: jest.fn().mockReturnThis() });
```

```
describe("roleMiddleware", () => {
  it("retorna 401 cuando falta req.usuario", () => {
    const mw = roleMiddleware("ADMIN");
    const res = fakeRes();
    mw({}, res, jest.fn());
    expect(res.status).toHaveBeenCalled();
  });
});
```

```
it("retorna 403 cuando el rol no está permitido", () => {
  const mw = roleMiddleware("ADMIN");
  const res = fakeRes();
  mw({ usuario: { rol: "MESERO" } }, res, jest.fn());
  expect(res.status).toHaveBeenCalled();
});
```

```
it("llama a next() cuando el rol es permitido", () => {
  const mw = roleMiddleware("ADMIN", "CAJA");
  const next = jest.fn();
  mw({ usuario: { rol: "CAJA" } }, fakeRes(), next);
  expect(next).toHaveBeenCalled();
});
});
```

authMiddleware y errorMiddleware siguen el mismo patrón: se provee una petición con o sin Authorization, se mockea jwt.verify, y se observa el estado y cuerpo de la respuesta, o next(err).

Apéndice B — Especificaciones de Pruebas Unitarias del Frontend

Los specs del frontend usan **Vitest** con **jsdom**.

B.1 FE-01 — Storage (frontend/utils/storage.js)

```
import { describe, it, expect, beforeEach } from "vitest";
import Storage from "../storage.js";

describe("Storage", () => {
  beforeEach(() => localStorage.clear());

  it("persiste y lee el token", () => {
    Storage.guardarToken("abc");
    expect(Storage.obtenerToken()).toBe("abc");
    expect(Storage.estaAutenticado()).toBe(true);
  });

  it("retorna null cuando no hay token", () => {
    expect(Storage.obtenerToken()).toBeNull();
    expect(Storage.estaAutenticado()).toBe(false);
  });

  it("persiste y lee el usuario", () => {
    Storage.guardarUsuario({ id: 1, rol: "ADMIN" });
    expect(Storage.obtenerUsuario()).toEqual({ id: 1, rol: "ADMIN" });
  });

  it("limpiarSesion elimina ambos", () => {
    Storage.guardarToken("abc");
    Storage.guardarUsuario({ id: 1 });
    Storage.limpiarSesion();
    expect(Storage.obtenerToken()).toBeNull();
    expect(Storage.obtenerUsuario()).toBeNull();
  });
});
```

B.2 FE-02 — Auth (frontend/utils/Auth.js)

```
import { describe, it, expect, beforeEach } from "vitest";
import Auth from "../Auth.js";
import Storage from "../storage.js";

describe("Auth", () => {
  beforeEach(() => localStorage.clear());

  it("delega autenticado en Storage", () => {
    expect(Auth.autenticado()).toBe(false);
    Storage.guardarToken("t");
    expect(Auth.autenticado()).toBe(true);
  });
});
```

```

});

it("rol retorna el rol del usuario actual", () => {
  Storage.guardarUsuario({ rol: "CAJA" });
  expect(Auth.rol()).toBe("CAJA");
});

it("rol retorna null cuando no hay usuario", () => {
  expect(Auth.rol()).toBeNull();
});

it("tieneRol coincide con uno de los roles listados", () => {
  Storage.guardarUsuario({ rol: "MESERO" });
  expect(Auth.tieneRol("ADMIN", "MESERO")).toBe(true);
  expect(Auth.tieneRol("ADMIN", "CAJA")).toBe(false);
});
});

```

B.3 FE-03 — EstadoPedido (frontend/utils/EstadoPedido.js)

```

import { describe, it, expect } from "vitest";
import EstadoPedido from "../EstadoPedido.js";

describe("EstadoPedido", () => {
  it("expone las constantes canónicas de estado", () => {
    expect(EstadoPedido.PENDIENTE).toBe("PENDIENTE");
    expect(EstadoPedido.COCINA).toBe("COCINA");
    expect(EstadoPedido.LISTO).toBe("LISTO");
    expect(EstadoPedido.ENTREGADO).toBe("ENTREGADO");
    expect(EstadoPedido.PAGADO).toBe("PAGADO");
  });
});

```

B.4 FE-04..08 — Validadores del Frontend

Mismo patrón para los cinco. Ejemplo para MesaValidator:

```

import { describe, it, expect } from "vitest";
import MesaValidator from "../MesaValidator.js";

describe("MesaValidator", () => {
  it("rechaza numero vacío", () => {
    expect(MesaValidator.validar({ numero: "", capacidad: 2 })).toMatchObject({ ok: false });
  });
  it("rechaza numero no positivo (BVA: 0)", () => {
    expect(MesaValidator.validar({ numero: 0, capacidad: 2 })).toMatchObject({ ok: false });
  });
});

```

```

it("rechaza capacidad no positiva (BVA: -1, 0)", () => {
  expect(MesaValidator.validar({ numero: 1, capacidad: 0 })).toMatchObject({ ok: false });
  expect(MesaValidator.validar({ numero: 1, capacidad: -1 })).toMatchObject({ ok: false });
});
it("acepta una mesa válida", () => {
  expect(MesaValidator.validar({ numero: 1, capacidad: 4 })).toEqual({ ok: true });
});

```

Espejos para:

- MenuValidator — nombre vacío, precio 0, negativo, positivo.
- ClienteValidator — nombre/teléfono vacíos.
- UsuarioValidator — campos vacíos, reglas de contraseña (rama con id vacío).
- PedidoValidator — firma nueva validar(items, canal, clienteNombre, telefonoEntrega, direccionEntrega):
 - items vacíos → {ok:false} "Debe agregar al menos un producto".
 - canal === "TELEFONO" con clienteNombre vacío → {ok:false} "nombre del cliente".
 - canal === "TELEFONO" con telefonoEntrega vacío → {ok:false} "teléfono de contacto".
 - canal === "TELEFONO" con direccionEntrega vacía → {ok:false} "dirección de entrega".
 - en otro caso ok.

B.5 FE-09 — MesaService sobre fetch (frontend/services/mesaService.js)

```

import { describe, it, expect, beforeEach, vi } from "vitest";
import MesaService from "../mesaService.js";
import Storage from "../utils/storage.js";

```

```

const jsonResponse = (body, status = 200) =>
  ({ ok: true, status, json: () => Promise.resolve(body) });

```

```

describe("MesaService", () => {
  beforeEach(() => {
    localStorage.clear();
    Storage.guardarToken("TOK3N");
    vi.restoreAllMocks();
  });

```

```

it("listar llama a GET /mesas con token bearer", async () => {
  const fetchMock = vi.fn().mockResolvedValue(jsonResponse({ success: true, data: [] }));
  vi.stubGlobal("fetch", fetchMock);
  await MesaService.listar();

```

```

const [url, init] = fetchMock.mock.calls[0];
expect(url).toBe("http://localhost:3000/api/v1/mesas");
expect(init.headers.Authorization).toBe("Bearer T0K3N");
});

it("crear envía POST con cuerpo JSON", async () => {
  const fetchMock = vi.fn().mockResolvedValue(jsonResponse({ success: true }));
  vi.stubGlobal("fetch", fetchMock);
  await MesaService.crear({ numero: 1, capacidad: 2 });
  const [url, init] = fetchMock.mock.calls[0];
  expect(init.method).toBe("POST");
  expect(init.body).toBe(JSON.stringify({ numero: 1, capacidad: 2 }));
});

it("actualizar usa PUT", async () => {
  const fetchMock = vi.fn().mockResolvedValue(jsonResponse({ success: true }));
  vi.stubGlobal("fetch", fetchMock);
  await MesaService.actualizar("m1", { numero: 2 });
  const [url, init] = fetchMock.mock.calls[0];
  expect(url).toContain("/mesas/m1");
  expect(init.method).toBe("PUT");
});

it("eliminar usa DELETE", async () => {
  const fetchMock = vi.fn().mockResolvedValue(jsonResponse({ success: true }));
  vi.stubGlobal("fetch", fetchMock);
  await MesaService.eliminar("m1");
  const [, init] = fetchMock.mock.calls[0];
  expect(init.method).toBe("DELETE");
});

it("cambiarEstado usa PATCH /mesas/:id/estado", async () => {
  const fetchMock = vi.fn().mockResolvedValue(jsonResponse({ success: true }));
  vi.stubGlobal("fetch", fetchMock);
  await MesaService.cambiarEstado("m1", "OCUPADA");
  const [url, init] = fetchMock.mock.calls[0];
  expect(url).toContain("/mesas/m1/estado");
  expect(init.method).toBe("PATCH");
  expect(init.body).toBe(JSON.stringify({ estado: "OCUPADA" }));
});

it("omite el encabezado Authorization cuando no hay token (corrección V3.0)", async () => {
  Storage.eliminarToken();
  const fetchMock = vi.fn().mockResolvedValue(jsonResponse({ success: true }));

```



```

vi.stubGlobal("fetch", fetchMock);
await MesaService.listar();
const [, init] = fetchMock.mock.calls[0];
expect(init.headers.Authorization).toBeUndefined();
});
});

```

B.6 FE-10..13 — api, Toast, Modal, EventBus

```

import { describe, it, expect } from "vitest";
import API from "./api.js";
describe("api", () => {
  it("expone la URL base v3 (render)", () => {
    expect(API).toBe("https://bocatto-2gaz.onrender.com/api/v1");
  });
});

import { describe, it, expect, vi, beforeEach } from "vitest";
import EventBus from "./EventBus.js";

describe("EventBus", () => {
  it("emitir/escuchar despacha eventos vía CustomEvent", () => {
    const cb = vi.fn();
    EventBus.escuchar("pedido:creado", cb);
    EventBus.emitir("pedido:creado", { id: 1 });
    expect(cb).toHaveBeenCalledTimes(1);
    expect(cb.mock.calls[0][0].detail).toEqual({ id: 1 });
  });

  it("eliminar quita un listener específico", () => {
    const cb = vi.fn();
    EventBus.escuchar("x", cb);
    EventBus.eliminar("x", cb);
    EventBus.emitir("x");
    expect(cb).not.toHaveBeenCalled();
  });
});

```

Toast y Modal se prueban importando bootstrap (devDependency) en jsdom y stubbeando bootstrap.Toast/bootstrap.Modal; se verifica la inserción/eliminación del elemento en el DOM y la invocación de show()/hide().

Apéndice C — Catálogo de Casos de Prueba

Cada entrada sigue la plantilla de especificación de caso de prueba de IEEE 829 (identificador, elementos, entradas, salidas, necesidades del entorno).

C.1 Casos de prueba del backend

TC ID	Elementos	Entradas	Resultado esperado
TC-UT-001	BE-01	new ApiError("x")	e instanceof Error, e.status === 500
TC-UT-002	BE-01	new ApiError("x", 404)	e.status === 404
TC-UT-003	BE-02	ApiResponse.success(res, "ok", {a:1})	res.status(200).json({success:true,message:"ok",data:{a:1}})
TC-UT-004	BE-02	ApiResponse.success(res, "ok", null, 201)	res.status(201).json(...)
TC-UT-005	BE-03	{nombre:"",apellido:"x",username:"y",password:"123456",rol:"ADMIN"}	lanza "nombre"
TC-UT-006	BE-03	{nombre:"a",apellido:"",...}	lanza "apellido"
TC-UT-007	BE-03	{...,password:"12345"}	lanza por longitud de contraseña
TC-UT-008	BE-03	{...,password:"123456",rol:"X"}	lanza "Rol inválido"
TC-UT-009	BE-04	{nombre:"", correo:"a@b.com"}	lanza "nombre"
TC-UT-010	BE-04	{nombre:"a", correo:"bad-format"}	lanza "Correo"
TC-UT-011	BE-05	{numero:1, capacidad:0}	lanza "Capacidad inválida"
TC-UT-012	BE-05	{numero:undefined, capacidad:2}	lanza "Número de mesa"
TC-UT-013	BE-06	{nombre:"", precio:1}	lanza "nombre"
TC-UT-014	BE-06	{nombre:"x", precio:0} / {nombre:"x", precio:-1}	lanza "precio debe ser mayor a cero"
TC-UT-015	BE-07	{canal:"MESA", items:[]}	lanza "Debe agregar al menos un producto"
TC-UT-016	BE-07	{canal:"MESA"} (sin mesa)	lanza "mesa"

TC ID	Elementos	Entradas	Resultado esperado
TC-UT-017	BE-07	{canal:"TELEFONO", items:[{cantidad:1}]} (sin clienteNombre / telefonoEntrega / direccionEntrega)	lanza "nombre del cliente" / "teléfono" / "dirección"
TC-UT-018	BE-07	{canal:"QR", items:[{cantidad:1}]} (sin clienteNombre)	lanza "QR requiere el nombre"
TC-UT-019	BE-07	{items:[{cantidad:0}]}	lanza "Cantidad inválida"
TC-UT-020	BE-08	{nombre:""}	lanza "Ingrese el nombre"
TC-UT-021	BE-08	{nombre:"x", descuento:-1} / 101	lanza "0 y 100"
TC-UT-022	BE-09	{_id:"x", numero:1, capacidad:2, estado:"LIBRE", extra:"ignore"}	{id:"x", numero:1, capacidad:2, estado:"LIBRE"} (sin extra)
TC-UT-023	BE-10	toDTOList([])	[]
TC-UT-024	BE-10	toDTOList([m1, m2])	longitud 2, los ítems son MesaDTO
TC-UT-025	BE-11	clon de {nested:{a:1}}	igualdad profunda, las referencias difieren
TC-UT-026	BE-12	clon de {tags:["a"]}	igualdad profunda, la referencia del arreglo difiere
TC-UT-027	BE-13	new Prototype().clone()	lanza "clone()"
TC-UT-028	BE-14	flujo completo del builder	acumula items, total, aplica descuento, valida, retorna copia
TC-UT-029	BE-14	agregarProducto(p, 0)	lanza "Cantidad inválida"
TC-UT-030	BE-14	agregarProducto({...p, precio:0}, 1)	lanza "Precio inválido"
TC-UT-031	BE-15	director con descuento	aplica el descuento correctamente
TC-UT-032	BE-16	obtenerMesaPorId inexistente	ApiError 404
TC-UT-033	BE-16	crearMesa con duplicado	ApiError 400

TC ID	Elementos	Entradas	Resultado esperado
TC-UT-034	BE-16	actualizarMesa mismo id y mismo numero	ok
TC-UT-035	BE-16	eliminarMesa inexistente	ApiError 404
TC-UT-036	BE-17	login con usuario desconocido	lanza "no existe"
TC-UT-037	BE-17	login con cuenta bloqueada dentro de la ventana	ApiError 403 BLOQUEADO:N
TC-UT-038	BE-17	login con ventana de bloqueo expirada	se llama desbloquearUsuario, continúa el flujo
TC-UT-039	BE-17	login con contraseña incorrecta (intentos 1, 2)	ApiError 401 INTENTOS:N; se llama actualizarIntentos
TC-UT-040	BE-17	login con 3er intento fallido	ApiError 403 BLOQUEADO:300; se llama bloquearUsuario
TC-UT-041	BE-17	login exitoso	se llama desbloquearUsuario, retorna token y usuario
TC-UT-042	BE-18	roleMiddleware sin req.usuario	401
TC-UT-043	BE-18	roleMiddleware con rol incorrecto	403
TC-UT-044	BE-18	roleMiddleware con rol correcto	se llama next()
TC-UT-045	BE-19	authMiddleware sin encabezado	401
TC-UT-046	BE-19	authMiddleware con encabezado válido	se llena req.usuario, se llama next()
TC-UT-047	BE-20	errorMiddleware recibe ApiError	renderiza el sobre JSON con el estado correcto

C.2 Casos de prueba del frontend

TC ID	Elementos	Entradas	Resultado esperado
TC-UT-101	FE-01	guardar/leer/eliminar token	round-trip ok; estaAutenticado refleja la presencia

TC ID	Elementos	Entradas	Resultado esperado
TC-UT-102	FE-01	guardar/leer/eliminar usuario	round-trip ok
TC-UT-103	FE-01	limpiarSesion	ambas claves eliminadas
TC-UT-104	FE-02	autenticado	delega en Storage
TC-UT-105	FE-02	rol	retorna el rol del usuario o null
TC-UT-106	FE-02	tieneRol	coincide con uno de los roles proporcionados
TC-UT-107	FE-03	constantes	cada constante equivale a su valor en string
TC-UT-108	FE-04	numero vacío	{ok:false}
TC-UT-109	FE-04	numero:0 / numero:-1	{ok:false}
TC-UT-110	FE-04	capacidad:0 / capacidad:-1	{ok:false}
TC-UT-111	FE-04	mesa válida	{ok:true}
TC-UT-112	FE-05	nombre vacío / precio 0 / precio -1	{ok:false}
TC-UT-113	FE-05	producto válido	{ok:true}
TC-UT-114	FE-06	items vacíos	{ok:false} "Debe agregar al menos un producto"
TC-UT-115	FE-06	TELEFONO sin clienteNombre	{ok:false} "nombre del cliente"
TC-UT-116	FE-06	TELEFONO sin telefonoEntrega	{ok:false} "teléfono de contacto"
TC-UT-117	FE-06	TELEFONO sin direccionEntrega	{ok:false} "dirección de entrega"
TC-UT-118	FE-06	válido	{ok:true}

TC ID	Elementos	Entradas	Resultado esperado
TC-UT-119	FE-07	nombre vacío	{ok:false} "nombre"
TC-UT-120	FE-07	teléfono vacío	{ok:false} "teléfono"
TC-UT-121	FE-07	válido	{ok:true}
TC-UT-122	FE-08	nombre / apellido / username vacíos	{ok:false}
TC-UT-123	FE-08	contraseña vacía al crear	{ok:false}
TC-UT-124	FE-08	válido	{ok:true}
TC-UT-125	FE-09	listar	GET, encabezado Authorization, URL https://bocatto-2gaz.onrender.com/api/v1/mesas
TC-UT-126	FE-09	crear	POST, cuerpo JSON
TC-UT-127	FE-09	actualizar	PUT
TC-UT-128	FE-09	eliminar	DELETE
TC-UT-129	FE-09	cambiarEstado	PATCH, cuerpo {estado}
TC-UT-130	FE-09	sin token	Authorization ausente (corrección V3.0)
TC-UT-131	FE-10	URL base	https://bocatto-2gaz.onrender.com/api/v1
TC-UT-132	FE-11	Toast.success("x")	el DOM contiene un elemento .toast con clase text-bg-success y el mensaje "x"
TC-UT-133	FE-12	Modal.abrir(id) / Modal.cerrar(id)	se invoca bootstrap.Modal.show() / hide()
TC-UT-134	FE-13	EventBus.emitir/escuchar	el listener se invoca con event.detail
TC-UT-135	FE-13	EventBus.eliminar	el listener no se invoca tras ser eliminado

Apéndice D — Extensión de Fase 2 (cobertura completa de RFs)

Esta extensión se añade tras la fase 1 para llevar la cobertura unitaria de 1/24 RFs a **23/24 RFs**. Los nuevos elementos y casos de prueba se listan a continuación. El RF 3.2.22 queda como brecha de implementación (sin módulo backend).

D.1 Nuevos elementos backend (BE-21..27)

ID	Elemento	Ruta
BE-21	PedidoService	backend/src/services/PedidoService.js (repositorio + modelos mockeados)
BE-22	MenuService	backend/src/services/MenuService.js (repositorio mockeado)
BE-23	PromocionService	backend/src/services/PromocionService.js (repositorio mockeado)
BE-24	ReporteService	backend/src/services/ReporteService.js (modelo Pedido mockeado)
BE-25	DashboardService	backend/src/services/DashboardService.js (modelos mockeados + helpers puros)
BE-26	UsuarioService	backend/src/services/UsuarioService.js (repositorio + bcrypt mockeados)
BE-27	ClienteService	backend/src/services/ClienteService.js (repositorio mockeado)

D.2 Nuevos elementos frontend (FE-14..24)

ID	Elemento	Ruta
FE-14	PedidoService (HTTP)	frontend/services/pedidoService.js
FE-15	CocinaService (HTTP)	frontend/services/cocinaService.js
FE-16	CajaService (HTTP)	frontend/services/cajaService.js
FE-17	MenuService (HTTP)	frontend/services/menuService.js
FE-18	PromocionesService (HTTP)	frontend/services/promocionesService.js
FE-19	ReporteService (HTTP)	frontend/services/reporteService.js
FE-20	DashboardService (HTTP)	frontend/services/dashboardService.js
FE-21	UsuarioService (HTTP)	frontend/services/usuarioService.js
FE-22	ClienteService (HTTP)	frontend/services/clienteService.js
FE-23	AuthService (HTTP, login)	frontend/services/authService.js
FE-24	qr-order-helpers (puros)	frontend/assets/js/qr-order-helpers.js

D.3 Nuevos casos de prueba — Backend (TC-UT-048..085)

TC ID	Elementos	Entradas / Escenario	Resultado esperado
TC-UT-048	BE-21	listarPedidos	delega en PedidoRepository.obtenerTodos

TC ID	Elementos	Entradas / Escenario	Resultado esperado
TC-UT-049	BE-21	buscarPorMesa("m1")	delega en obtenerPorMesaActiva
TC-UT-050	BE-21	obtenerPedido("x") inexistente	ApiError 404
TC-UT-051	BE-21	crearPedido MESA válido	marca mesa OCUPADA
TC-UT-052	BE-21	crearPedido TELEFONO válido	ok, sin invocar Mesa.findById
TC-UT-053	BE-21	crearPedido QR sin mesa + clienteNombre	ok, genera codigoSeguimiento BC-...
TC-UT-054	BE-21	crearPedido con promoción	aplica el descuento al total
TC-UT-055	BE-21	crearPedidoQR("mQ", datos)	canal=QR, mesa="mQ"
TC-UT-056	BE-21	obtenerSeguimientoPorCodigo("BC-..") existente	retorna forma pública sin _id
TC-UT-057	BE-21	obtenerSeguimientoPorCodigo("NOPE")	ApiError 404
TC-UT-058	BE-21	cambiarEstado("p1","LISTO")	ok, no libera mesa
TC-UT-059	BE-21	cambiarEstado("p1","PAGADO")	ok, libera mesa (Mesa.findByIdAndUpdate("m1",{estado:"LIBRE"}))
TC-UT-060	BE-21	cambiarEstado("p1","ENTREGADO")	ok, no libera mesa
TC-UT-061	BE-21	cambiarEstado("x","LISTO") inexistente	ApiError 404
TC-UT-062	BE-21	pagarPedido("p1","EFECTIVO") pedido LISTO	ok, estado PAGADO, libera mesa
TC-UT-063	BE-21	pagarPedido("p1") ya PAGADO	ApiError 400 "ya fue pagado"
TC-UT-064	BE-21	pagarPedido("x") inexistente	ApiError 404
TC-UT-065	BE-21	generarCodigoSeguimiento (100 llamadas)	todos únicos, prefijo BC-, 3 partes

TC ID	Elementos	Entradas / Escenario	Resultado esperado
TC-UT-066	BE-22	MenuService.listarMenu	delega en obtenerTodos
TC-UT-067	BE-22	MenuService.listarDisponibles	delega en obtenerDisponibles
TC-UT-068	BE-22	MenuService.crearProducto	delega en crear
TC-UT-069	BE-22	MenuService.actualizarProducto	delega en actualizar
TC-UT-070	BE-22	MenuService.eliminarProducto	delega en eliminar
TC-UT-071	BE-22	MenuService.obtenerProducto	delega en obtenerPorId
TC-UT-072	BE-25	obtenerGrafico con pedidos PAGADO lunes/domingo	mapea al día de la semana correcto
TC-UT-073	BE-25	obtenerGrafico con pedidos no PAGADO	valores en 0
TC-UT-074	BE-25	productosMasVendidos con varios items	agrega y ordena descendentemente, top 5
TC-UT-075	BE-25	productosMasVendidos con items vacíos	retorna []
TC-UT-076	BE-25	resumen con modelos mockeados	ventas, pedidos activos, mesas ocupadas, clientes correctos
TC-UT-077	BE-25	obtenerGrafico([])	7 ceros
TC-UT-078	BE-23	PromocionService.listarPromociones	delega en obtenerTodas
TC-UT-079	BE-23	PromocionService.listarActivas	delega en obtenerActivas
TC-UT-080	BE-23	PromocionService.crear/actualizar/eliminar	delega en los métodos del repositorio
TC-UT-081	BE-23	PromocionService.obtenerPromocion	delega en obtenerPorId
TC-UT-082	BE-23	todas las operaciones	todas las CRUD delegan correctamente

TC ID	Elementos	Entradas / Escenario	Resultado esperado
TC-UT-083	BE-24	obtenerVentas(inicio, fin)	filtro con estado.\$in + createdAt.\$gte/\$lte
TC-UT-084	BE-24	obtenerVentas() sin fechas	filtro solo por estado
TC-UT-085	BE-24	cadena populate("mesa").populate("cliente").sort({createdAt:-1})	se invoca en orden

D.4 Nuevos casos de prueba — Frontend (TC-UT-136..170)

TC ID	Elementos	Entradas / Escenario	Resultado esperado
TC-UT-136	FE-14	pedidoService.listar/crear/actualizar/buscarPorMesa	verbos y URLs correctas, body JSON, Authorization condicional
TC-UT-137	FE-14	pedidoService.buscarPorMesa("m1")	GET /pedidos/mesa/m1
TC-UT-138	FE-14	sin token	Authorization ausente
TC-UT-139	FE-15	cocinaService.listar	GET /pedidos con token
TC-UT-140	FE-15	cocinaService.actualizarEstado("p1","COCINA")	PUT /pedidos/p1 body {estado:"COCINA"}
TC-UT-141	FE-15	sin token	Authorization ausente
TC-UT-142	FE-16	cajaService.cobrar("p1","TARJETA")	PUT /pedidos/p1/pagar body {metodoPago:"TARJETA"}
TC-UT-143	FE-16	cajaService.entregar("p1")	PUT /pedidos/p1 body {estado:"ENTREGADO"}
TC-UT-144	FE-17	menuService.listar/obtener/crear/actualizar/eliminar	CRUD correcto sobre /menu
TC-UT-145	FE-17	sin token	Authorization ausente
TC-UT-146	FE-20	dashboardService.obtenerResumen	GET /dashboard/resumen con token
TC-UT-147	FE-18	sin token	Authorization ausente
TC-UT-148	FE-18	promocionesService.listar/obtener/crear/actualizar/eliminar	CRUD correcto sobre /promociones

TC ID	Elementos	Entradas / Escenario	Resultado esperado
TC-UT-149	FE-18	sin token	Authorization ausente
TC-UT-150	FE-19	reporteService.listar("2026-01-01","2026-01-31")	GET /reportes?inicio=...&fin=...
TC-UT-151	FE-21	usuarioService.listar/obtener/crear/actualizar/eliminar	CRUD correcto sobre /usuarios
TC-UT-152	FE-22	clienteService.listar/obtener/crear/actualizar/eliminar	CRUD correcto sobre /clientes
TC-UT-153	FE-21	sin token	Authorization ausente
TC-UT-154	FE-22	sin token	Authorization ausente
TC-UT-155	FE-19	sin token	Authorization ausente
TC-UT-156	FE-20	sin token	Authorization ausente
TC-UT-157	FE-14	pedidoService.crear body	POST con cuerpo JSON correcto
TC-UT-158	FE-23	authService.login("admin","123456")	POST /auth/login, body {username,password}, sin Authorization
TC-UT-159	FE-23	authService.login error response	propaga el JSON {success:false,message}
TC-UT-160	FE-24	moneda(2.5) / moneda(null)	\$2.50 / \$0.00
TC-UT-161	FE-24	totalItems([]) / totalItems([{cantidad:2,precio:2.5}])	0 / 5
TC-UT-162	FE-24	estadoBadgeClass para cada estado	mapea a clase Bootstrap correcta
TC-UT-163	FE-24	agregarProductoACarrito([], producto)	agrega con cantidad 1
TC-UT-164	FE-24	agregarProductoACarrito con producto existente	incrementa cantidad
TC-UT-165	FE-24	agregarProductoACarrito con producto nulo	no muta el carrito

<i>TC ID</i>	<i>Elementos</i>	<i>Entradas / Escenario</i>	<i>Resultado esperado</i>
TC-UT-166	FE-24	cambiarCantidadEnCarrito +1 / -1	incrementa / decrementa
TC-UT-167	FE-24	cambiarCantidadEnCarrito a 0	elimina el item
TC-UT-168	FE-24	quitarDeCarrito(items, 0)	elimina sin mutar el original
TC-UT-169	FE-24	ESTADO_MENSAJES para COCINA/LISTO/ENTREGADO/PAGADO	contiene los mensajes esperados
TC-UT-170	FE-24	ESTADO_TIPOS para cada estado	tipos de toast correctos

D.5 Production fixes aplicados en fase 2

1. backend/src/services/UsuarioService.js — añadido import ApiError (D-03).
2. backend/src/validators/PedidoValidator.js y backend/src/patterns/builder/PedidoBuilder.js — el canal QR ya no exige mesa (D-04).
3. frontend/assets/js/qr-order.js — extracción de helpers puros a qr-order-helpers.js (D-05).

Fin del documento.